Bitwise Operations

You are implementing a simple **bit-packing** routine. You have:

```
unsigned int buffer = 0; int totalBits = 0;
```

You receive three values to pack sequentially into buffer starting from the least-significant end:

Value	Number of valid bits
val1 = 0x5	bits1 = 3
val2 = 0xF	bits2 = 4
val3 = 0x2	bits3 = 2

(a) Write C code to pack all three values into buffer (val1 at the bottom, val2 above it, val3 above that).

```
unsigned int buffer = 0;
int totalBits = 0;

buffer = val1;
buffer |= val2 << bits1;
buffer |= val3 << (bits1 + bits2);

totalBits = bits1 + bits2 + bits3;
```

C Pointers and Memory

Consider the following declarations:

```
char *words[4];
char buf[] = "hello world";
```

(a) What is the difference between char *words[4] and char (*words)[4]?

`char *words[4];` means `words` is an array of 4 pointers to `char`.
`char (*words)[4];` means `words` is a pointer to an array of 4 chars.

(b) After words[0] = buf;, what does words[0][3] evaluate to?

`'l'`
because `"hello world"` has:

```
h e l l o
0 1 2 3 4
```

(c) Write code using malloc and strcpy to make words[1] point to its own independent copy of "goodbye".

```
words[1] = malloc(strlen("goodbye") + 1);
strcpy(words[1], "goodbye");
```

argc / argv and String Handling

A program is invoked as:

```
./mygrep -i -n "hello world" data.txt
```

(a) What are the values of argc and each element of argv?

argc = 5;

argv[0] = "./mygrep";
argv[1] = "-i";
argv[2] = "-n";
argv[3] = "hello world";
argv[4] = "data.txt";
argv[5] = NULL;

(b) Why is comparing an argument to a string using if (argv[1] == "-i") incorrect? Explain and show the correct way.

because == compares pointer addresses, not string contents.

```
if (strcmp(argv[1], "-i") == 0) {
   printf("ignore case option found\n");
}
```

Process Control and wait

```
pid_t pid = fork();
if (pid == 0) {
     execlp("ls", "ls", "-l", NULL);
     perror("exec failed");
     exit(1);
}
int status;
wait(&status);
```

(a) Using the macros WIFEXITED, WEXITSTATUS, WIFSIGNALED, and WTERMSIG, write code that prints either the exit status or the signal number that terminated the child.

```
if (WIFEXITED(status)) {
   printf("child exited with status %d\n", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
   printf("child terminated by signal %d\n", WTERMSIG(status));
}
```

(b) Explain what happens if you call exec and it **fails**. Why is the perror / exit after execlp important?

If `execlp()` succeeds, the current process is replaced by `ls`, so the lines after `execlp()` do not run.

If `execlp()` fails, the child continues executing the next lines.

(c) What happens if the parent never calls wait? What is the child process called in that state?

If the parent never calls `wait()`, the child may become a **zombie process** after it exits.

A zombie has finished running, but its exit status has not yet been collected by the parent.

Unix I/O Redirection with System Calls

A shell needs to execute the command: sort < input.txt > output.txt

(a) List, **in order**, the system calls the shell makes (after fork, in the child) to set up the redirections and run sort. Be specific about arguments to open and dup2.

```
int in = open("input.txt", O_RDONLY);
dup2(in, STDIN_FILENO);
close(in);

int out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(out, STDOUT_FILENO);
close(out);

execlp("sort", "sort", NULL);
perror("exec sort failed");
exit(1);
```

(b) Why must the original file descriptors from open be closed after calling dup2?

After `dup2()`, standard input or standard output now refers to the file. The original descriptor from `open()` is no longer needed.

Closing it prevents file descriptor leaks.

Pipes

A shell needs to execute: cat file.txt | grep "error" | wc -l

(a) How many calls to pipe() are needed? How many calls to fork()?

2 calls to pipe()
3 calls to fork()

(b) Describe what each process does with the pipe file descriptors (which ends it dups and which it closes).

pipe1 connects cat -> grep
pipe2 connects grep -> wc
---
cat file.txt:

stdout goes to pipe1 write end
closes unused pipe ends
execs cat

---
grep "error":

stdin comes from pipe1 read end
stdout goes to pipe2 write end
closes unused pipe ends
execs grep

---
wc -l:

stdin comes from pipe2 read end
closes unused pipe ends
execs wc

(c) What happens if the writing end of a pipe is never closed in the reading process? Why does this matter?

If a write end of a pipe remains open, the reader may never see EOF.

For example, `wc -l` may keep waiting because it thinks more input could still arrive.

Signals

(a) Explain the difference between signal() and sigaction(). Give one specific capability that sigaction() provides that signal() does not.

`signal()` installs a basic signal handler.

`sigaction()` is more reliable and gives more control. For example, `sigaction()` can:

set flags such as SA_RESTART
control which signals are blocked while the handler runs
use SA_SIGINFO to get extra information about the signal

(b) Write a short program that sets up a handler for SIGALRM, calls alarm(5), then calls pause(). Explain what happens step by step.

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handler(int sig) {
    printf("alarm received\n");
}

int main(void) {
    signal(SIGALRM, handler);

    alarm(5);
    pause();

    printf("pause returned\n");
    return 0;
}
```

(c) Can SIGKILL be caught or ignored? Explain why or why not from a system-design perspective.

No. `SIGKILL` cannot be caught, ignored, or handled.

This gives the operating system a guaranteed way to terminate a process.

Unix Commands and the Shell

(a) Give a single shell pipeline that counts how many .c files exist anywhere under your home directory.

```
find ~ -name "*.c" | wc -l
```

(b) Explain the difference between Ctrl-C and Ctrl-Z in terms of which signal each sends and what the default behavior is.

`Ctrl-C` sends `SIGINT`.

`terminate the foreground process`

`Ctrl-Z` sends `SIGTSTP`.

`stop/suspend the foreground process`

(c) You have a file myscript.sh. You type ./myscript.sh and get Permission denied. Show **two** different ways to still execute it.

```
bash myscript.sh
```

```
chmod +x myscript.sh
./myscript.sh
```

make and Compilation

Given this Makefile:

```
myapp: main.o utils.o
      gcc -o myapp main.o utils.o

main.o: main.c defs.h
      gcc -c main.c

utils.o: utils.c defs.h
      gcc -c utils.c
```

(a) If you modify only defs.h, which targets get rebuilt and why?

If only `defs.h` changes, these targets are rebuilt:

```
main.o
utils.o
myapp
```

Why:

```
main.o depends on defs.h
utils.o depends on defs.h
myapp depends on main.o and utils.o
```

(b) How does make decide whether a target needs to be rebuilt? Be specific about what it compares.

`make` compares timestamps.

A target is rebuilt if: the target does not exist or any dependency is newer than the target

(c) If you type make and everything is up to date, then you run touch utils.c and type make again, what gets recompiled?

utils.o
myapp

Threads and Synchronization

```
int counter = 0;
void *increment(void *arg) {
     for (int i = 0; i < 1000000; i++)
         counter++;
     return NULL;
}
```

(a) If two threads both run increment, will counter reliably equal 2,000,000 at the end? Explain why or why not.

No.

`counter++` is not atomic. It usually involves: read counter, add 1, write counter

Two threads can interleave these steps and lose updates, causing the final value to be less than `2,000,000`.

(b) Fix the code using a **pthread mutex**. Show the declaration, initialization, and where lock/unlock calls go.

```
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg) {
  for (int i = 0; i < 1000000; i++) {
    pthread_mutex_lock(&lock);
    counter++;
    pthread_mutex_unlock(&lock);
  }

  return NULL;
}
```

(c) Why do threads need locks but separate processes (with separate address spaces) typically do not?

Threads need locks because they share the same address space and can access the same variables at the same time. Separate processes usually have separate address spaces, so one process's normal variables are not directly shared with another process.